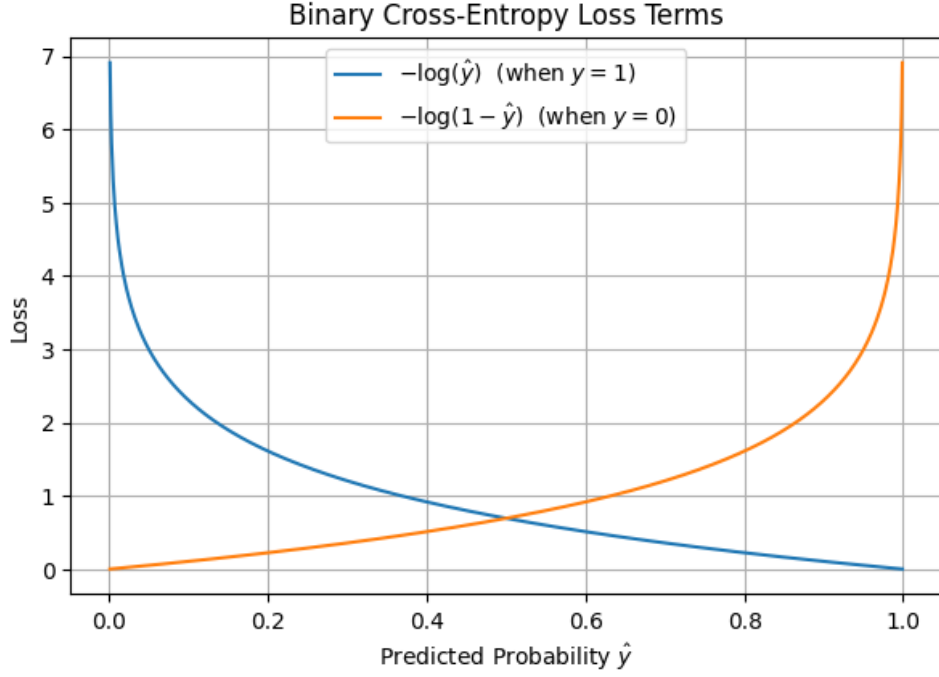




1 Gradient Derivation

1.1 Step 1: Differentiate BCE Loss

Starting with

$$\ell = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})], \quad (1)$$

differentiate with respect to $\hat{y}$:

$$\frac{\partial \ell}{\partial \hat{y}} = -\left(\frac{y}{\hat{y}} - \frac{1 - y}{1 - \hat{y}}\right). \quad (2)$$

1.2 Step 2: Differentiate Sigmoid

Since

$$\hat{y} = \sigma(z),$$

the derivative is

$$\frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y}). \quad (3)$$

1.3 Step 3: Apply Chain Rule

Using

$$\frac{\partial \ell}{\partial z} = \frac{\partial \ell}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z}, \quad (4)$$

we obtain

$$\frac{\partial \ell}{\partial z} = - \left(\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}} \right) \hat{y}(1-\hat{y}) \quad (5)$$

$$= \hat{y} - y. \quad (6)$$

Therefore,

$$\boxed{\frac{\partial \ell}{\partial z} = \hat{y} - y} \quad (7)$$

which is the prediction error.

1.4 Step 4: Differentiate Linear Model

Recall

$$z = \mathbf{w}^T \mathbf{x} + b. \quad (8)$$

Differentiating with respect to $\mathbf{w}$ gives

$$\frac{\partial z}{\partial \mathbf{w}} = \mathbf{x}. \quad (9)$$

Applying the chain rule once more,

$$\nabla_{\mathbf{w}} \ell = \frac{\partial \ell}{\partial z} \frac{\partial z}{\partial \mathbf{w}} \quad (10)$$

$$= (\hat{y} - y) \mathbf{x}. \quad (11)$$

Hence,

$$\boxed{\nabla_{\mathbf{w}} \ell = (\hat{y} - y) \mathbf{x}} \quad (12)$$

and

$$\boxed{\frac{\partial \ell}{\partial b} = \hat{y} - y.} \quad (13)$$

2 Gradient Descent Update Rule

The weight update equation becomes

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta(\hat{y} - y) \mathbf{x}. \quad (14)$$

Similarly,

$$b^{(t+1)} = b^{(t)} - \eta(\hat{y} - y). \quad (15)$$

The gradient consists of two components:

$$(\hat{y} - y) \mathbf{x}. \quad (16)$$

- $(\hat{y} - y)$ represents prediction error.
- $\mathbf{x}$ scales the update according to feature values.

Therefore,

- Large prediction error produces large updates.
- Small prediction error produces small updates.

3 Summary

The Logistic Regression model is

$$z = \mathbf{w}^T \mathbf{x} + b, \quad (17)$$

$$\hat{y} = \sigma(z), \quad (18)$$

$$\ell = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]. \quad (19)$$

The final gradients are

$$\boxed{\nabla_{\mathbf{w}} \ell = (\hat{y} - y) \mathbf{x}} \quad (20)$$

and

$$\boxed{\frac{\partial \ell}{\partial b} = \hat{y} - y.} \quad (21)$$

These equations form the foundation of Logistic Regression and modern Neural Network training using Gradient Descent and Backpropagation.

4 Gradient Derivation for Categorical Cross-Entropy

Consider a multiclass classification problem with K classes.

The output layer produces logits

$$\mathbf{z} = [z_1, z_2, \dots, z_K]^T. \quad (22)$$

The Softmax activation converts logits into probabilities:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \quad i = 1, 2, \dots, K. \quad (23)$$

The target vector is one-hot encoded:

$$\mathbf{y} = [y_1, y_2, \dots, y_K]^T,$$

where

$$y_i = \begin{cases} 1, & \text{correct class} \\ 0, & \text{otherwise.} \end{cases}$$

4.1 Step 1: Categorical Cross-Entropy Loss

The loss function is

$$L = - \sum_{i=1}^K y_i \log(\hat{y}_i). \quad (24)$$

For example, if the true class is class 3, then

$$\mathbf{y} = [0, 0, 1, 0, \dots, 0]^T,$$

and

$$L = -\log(\hat{y}_3).$$

4.2 Step 2: Differentiate Loss with Respect to Predictions

Differentiating with respect to $\hat{y}_i$,

$$\frac{\partial L}{\partial \hat{y}_i} = -\frac{y_i}{\hat{y}_i}. \quad (25)$$

4.3 Step 3: Differentiate Softmax Function

The softmax output is

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}. \quad (26)$$

Its Jacobian is

$$\boxed{\frac{\partial \hat{y}_i}{\partial z_j} = \hat{y}_i(\delta_{ij} - \hat{y}_j)} \quad (27)$$

where

$$\delta_{ij} = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

4.4 Step 4: Apply Chain Rule

Using

$$\frac{\partial L}{\partial z_j} = \sum_{i=1}^K \frac{\partial L}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial z_j}, \quad (28)$$

we obtain

$$\frac{\partial L}{\partial z_j} = \sum_{i=1}^K \left(-\frac{y_i}{\hat{y}_i}\right) \hat{y}_i(\delta_{ij} - \hat{y}_j) \quad (29)$$

$$= \sum_{i=1}^K -y_i(\delta_{ij} - \hat{y}_j) \quad (30)$$

$$= \hat{y}_j - y_j. \quad (31)$$

Therefore,

$$\boxed{\frac{\partial L}{\partial z_j} = \hat{y}_j - y_j} \quad (32)$$

or equivalently, in vector form,

$$\boxed{\nabla_{\mathbf{z}} L = \hat{\mathbf{y}} - \mathbf{y}} \quad (33)$$

which represents the prediction error vector.

4.5 Step 5: Gradient with Respect to Weights

Recall that

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}. \quad (34)$$

Differentiating,

$$\frac{\partial \mathbf{z}}{\partial W} = \mathbf{x}. \quad (35)$$

Applying the chain rule,

$$\boxed{\frac{\partial L}{\partial W} = (\hat{\mathbf{y}} - \mathbf{y})\mathbf{x}^T} \quad (36)$$

and

$$\boxed{\frac{\partial L}{\partial \mathbf{b}} = \hat{\mathbf{y}} - \mathbf{y}} \quad (37)$$

5 Summary

For Binary Cross-Entropy with Sigmoid,

$$\boxed{\nabla_{\mathbf{w}} \ell = (\hat{y} - y)\mathbf{x}} \quad (38)$$

For Categorical Cross-Entropy with Softmax,

$$\boxed{\frac{\partial L}{\partial W} = (\hat{\mathbf{y}} - \mathbf{y})\mathbf{x}^T} \quad (39)$$

Thus, in both cases, the gradient takes the elegant form

$$\boxed{\text{Gradient} = \text{Prediction} - \text{Target}} \quad (40)$$

which is one of the key reasons why Sigmoid+BCE and Softmax+CCE are widely used in deep learning.